78%.

EIE 1st year Project

Group no 5 - Real Time Phrase Detection - Chia Hossain Lua

by Mehedi Hossain

FILE	GROUP_NO_5_-_REAL_TIME_PHRASE_DETECTION_-_CHIA_HOSSAIN_LUA.PDF (2.89M)		
TIME SUBMITTED	19-MAY-2017 08:15PM	WORD COUNT	8935
SUBMISSION ID	73010035	CHARACTER COUNT	45225

Imperial College London

TECHNICAL REPORT

Mehedi Hossain
EIE1
CID: 01219041
mh3216@ic.ac.uk

Patrick John Chia
EIE1
CID: 01259148
pjc316@ic.ac.uk

Ying Hao Lua
EIE1
CID: 01044210
yhl15@ic.ac.uk

Engineering Design & Practice
Group Project

Due Date: 19/05/17

Word Count: 7962

ABSTRACT

This report proposes a design to perform phrase detection on fonts with different physical attributes. Exploiting the resources and the capabilities of the FPGA, the design was optimised to deliver a smooth video output. Some key techniques employed included the use of a double buffer as well as the 'HLS Dataflow' type.

Author Keywords

Vivado HLS, SDSoC, FPGA, Hardware Acceleration, Image Processing, Amplitude Segmentation, Double Buffering, Dataflow directive, Pipelining, Arbitrary Precision Types.

CONTENTS

Abstract ii

Author Keywords ii

1. Updated Project Description ..... Pg. 3

2. Progress Since Interim Report

2.1 High Level Description ..... Pgs. 3-4

2.2 Project Planning and Management ..... Pgs. 4-5

3. Design Specification ..... Pg. 6

4. Design Process

4.1 Pre-Processing ..... Pg. 7

4.2 Phrase Outlining ..... Pgs. 7-8

4.3 Terminal User Interface ..... Pg. 8

4.4 Hardware Optimisations ..... Pg. 9

5. Proposed Solutions

5.1 Pre-Processing ..... Pg. 10

5.2 Phrase Detection ..... Pgs. 10-12

5.3 Phrase Outline Printing ..... Pg. 12

5.4 Terminal User Interface ..... Pg. 12

6. Final Design

6.1 Background and Considerations ..... Pg. 13

6.2 Final Design Overview ..... Pgs. 14-15

6.2.1 Phrase Detection ..... Pg. 15

6.2.2 Video Output Generation ..... Pg. 15

6.3 Design Rationale ..... Pgs. 16-19

6.3.1 Pipelining ..... Pg. 16

6.3.1.1 Fine Grain ..... Pg. 16

6.3.1.2 Coarse Grain ..... Pg. 17

6.3.2 Double Buffer ..... Pgs. 17-18

6.3.3 Arbitrary Precision Types ..... Pg. 19

6.3.4 Summary of Design Choices ..... Pg. 19

7. Outcomes

7.1 Performance ..... Pg. 20

7.2 Resource Usage ..... Pg. 20

7.3 Accuracy and Reliability ..... Pg. 21

7.4 Limitations ..... Pgs. 21-22

7.5 Suggestions for the Future ..... Pg. 23

7.6 Conclusions ..... Pg. 23

References ..... Pg. 24

Appendix A: Code for Final Design

A.1 System Setup and Control ..... Pgs. 25-27

A.2 Coarse Grain Pipelining ..... Pgs. 28-29

A.3 Sub-system Functions ..... Pgs. 30-35

1. UPDATED PROJECT DESCRIPTION

Initially, the group envisioned a project which aimed to capture a frame of dish names on a food menu, and overlay an image of the dish selected. With consideration of the limited space on a restaurant's menu, the team thought it was a fantastic idea as demand for such a service was identified, through surveying peers. The group expected the project to be complicated from the start, as it required stitching together several complex sub-systems. It involved phrase detection of sentences on a menu, then segmenting each individual character of the selected phrase. The next section involved sending the data obtained from each character through a custom Artificial Neural Network (ANN). The ANN would then output the "string" interpretation of the word, allowing an image of the dish to be fetched from the internet, and then overlaid on the screen.

After feedback from the project coordinator, the team realised that meeting the specific requirements of real time image processing on such a complex task was too ambitious, given the limited amount of time available. Thus, the team decided to narrow down the scope and develop an extremely responsive version of the first sub-system to detect a phrase. Furthermore, to validate the phrase recognition capabilities, the phrases were boxed up and overlaid on the output to the screen.

With that in mind, the new aim of the project was to detect phrases of variable size, colour, and font on mono - coloured backgrounds. Once detected, the system would highlight the phrases by outlining the edges at processing speeds that result in a smooth video output.

2. PROGRESS SINCE INTERIM REPORT

2.1 High Level Description

A video feed from an external source would be sent to the ZYBO board via HDMI. The board then processed the current frame in the video before it sent the resulting frame to a monitor via VGA output. The processing detected phrases on a line, and outlined the edges; the result was that each horizontal line containing words would be considered as one phrase. Each frame processed could contain many lines, and therefore many phrases.

Below, in Figure 2.1.1, is a block view of how the information flows in from the external 'camera', to being sent to the external 'display'. The 'ZYBO Board' is responsible for converting the input format to the output format, as well any intentional processing to alter the output video feed.

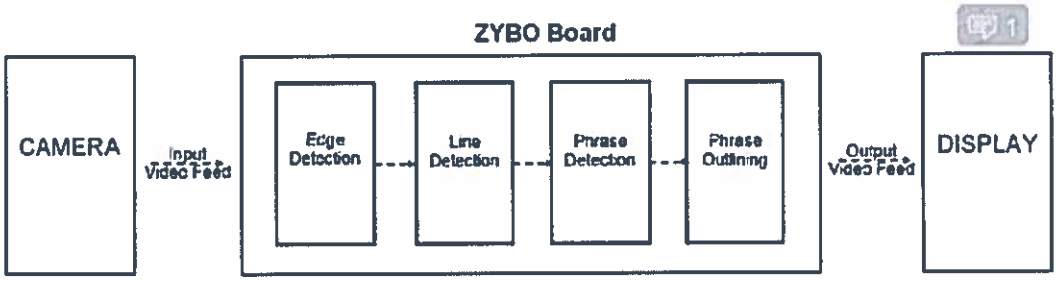


Figure 2.1.1: High Level Schematic

There were four stages to processing each frame, the first was to detect edges, as edges are key features making up the characters in the phrases. Once the edges were detected, the team had to develop an algorithm that would detect the lines that would contain the phrases. Next, the team had to design a function to find the phrases within the detected lines. The final function to create, involved outlining the phrases the system had detected, by injecting black lines along the top, bottom, right and left borders of the phrase. Figure 2.1.2 displays the group's aim in outlining a phrase.

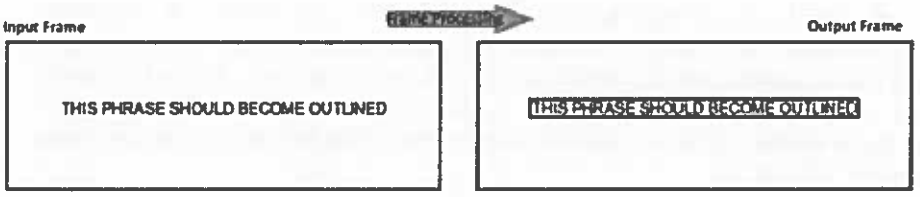


Figure 2.1.2 How a Phrase Should be Outlined

The system relied on the FPGA to process the frames and give a high throughout. Sub-systems running on the FPGA included the video drivers as well as the frame processing functions. Figure 2.1.3 gives a block view of the top-level functions.

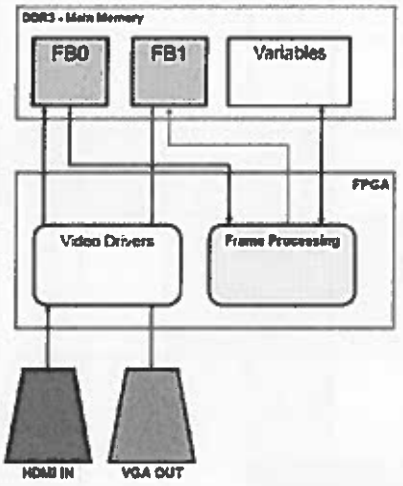


Figure 2.1.3 Flow of the Video Stream

'Video Drivers' were run on the FPGA, as shown in Figure 2.1.3; the first of which, converted the 'HDMI IN' signal to an array of Red, Green and Blue, (RGB) values stored in a Frame Buffer 'FB0', (detail on the data structure used is available in the management report). The 'Frame Processing' stage, running on the FPGA, would stream the pixels from 'FB0' and stream processed pixels to 'FB1'. Both Frame Buffers used were stored on the main board memory. Another 'Video Driver' extracted the RGB values for pixels from 'FB1' and converted them to a 'VGA' output format.

2.2 Project Planning and Management

Role	Description of Responsibilities	Person in charge
Project Management	Ensure smooth delivery of project milestones by meeting targets with adherence to deadlines agreed on the Gantt Chart. Manage the transition of concepts from high level implementation down to the hardware implementation on the FPGA Board. Keep track of sessions on Concept Generation	Ying Hao Lua Mehedi Hossain
Hardware Engineer	Research optimisations for HW acceleration Learn tools available on Vivado HLS and SDSoC	Patrick John Chia Yi Hean Mehedi Hossain
Software Engineer	Implement algorithms researched for required sub-systems Develop a solution for data to match the outlined information flow	Patrick John Chia Yi Hean Ying Hao Lua

As the group were working through the tasks, it was realised that each member was contributing towards nearly all the tasks, so the responsibilities of each role were updated. This was a switch from the initial plan to members working individually on tasks, working as a unit improved the team's efficiency at each task, as potential issues were spotted earlier by having three minds focus on a problem. Before coming together to collaborate, each member briefed the team on the progress made on their initilly assigned task.

Legend of Responsibilities:
- Entire Team
- Project Manager
- Software Engineer
- Hardware Engineer

Project Milestones/ Deliverables	Spring Term				Easter Break					Summer Term		
	27/2 - 5/3	6/3 – 12/3	13/3 – 19/3	20/3 – 26/3	27/3 – 2/4	3/4 – 9/4	10/4 – 16/4	17/4 – 23/4	24/4 – 30/4	1/5 – 7/5	8/5 – 14/5	15/5 – 19/5
Familiarisation of tools in Lab sessions												
Designing Project Scope												
Proof of Concept and High-level implementation												
Submission of Management Report												
Hardware Implementation on FPGA board												
Combining modules and completion of Prototype												
Analysis and Evaluation to improve existing design												
Finalise Design												
Preparation of Final Report (Outcomes and Process).												
Preparation for oral presentation												

3. DESIGN SPECIFICATION

The critical target of the project was to achieve a minimum video output of 25 FPS at a resolution of 1080p, while performing the phrase detection. The group had not set a hard limit on latency, but agreed on the speciation of 'un-hindering to the user experience'. This meant that the latency had to be good enough for the user to not to feel discomfort while using the system.

The design was expected to be used indoors, preferably at a computer desk, as the ZYBO board would have to be connected to a computer and an external monitor. Operating duration was only tested at a maximum of 30 minutes, so the group advise not to run the system for a longer period as system behaviour has not been tested over 30 minutes, the user should reset the board for extended use.

Intended use for the design was for demonstrative purposes only, the system was not expected to be launched and sold. The system was designed to only work with a pair of any two colours, one for the font and one for the background. Testing was carried out under controlled lighting conditions, and colours of background and font were chosen by the group.

The system was to have three main phrases during operation; initialisation, idle and processing, as shown in Figure 3.1. The initialisation would involve booting into the chosen OS, and to execute all the initialisation functions for the platform. When the system had entered idle, the device would act as a video pass-through. Finally, the processing stage should have been designed to have two modes, accelerated and CPU only operation. In the accelerated mode, the FPGA should be used to speed up the processing of the video stream, and in the CPU only mode, all functions are executed on the CPU.

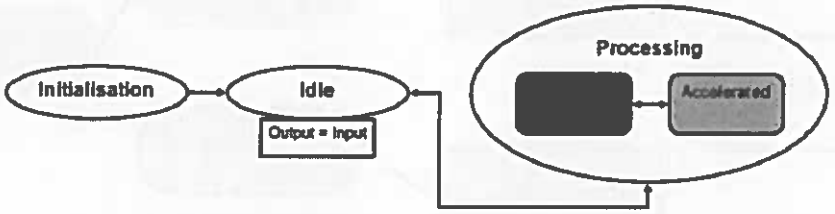


Figure 3.1: State Diagram of the System

A software FPS measurement must be included, to objectively compare the performance of the accelerated mode vs CPU only mode. The output of this measurement was to be sent to the console of the computer controlling the ZYBO Board, over UART.

The team had to eliminate any possibilities of flashing images or overly bright frames, suddenly appearing on the output video stream. To prevent this, extensive testing would have to be performed for stability, alongside checking for any bugs in the code that could give rise to such a situation. The group also highly advise to not touch the board whilst in operation, especially the SoC, as it does get very hot whilst the system is powered on, (and the FPGA is actively processing data).

The 'Product Design Specification' method, (PDS), was used to generate this specification, the other points of PDS do no not apply to this project so have not been used.

4. DESIGN PROCESS

4.1 Pre-Processing

The first sub-system needed to convert the input frame to a standard with which the later sub-systems could work from. After discussing about choosing a standard form, the group agreed to apply the 'SCAMPER' method (3) to the Sobel Filter (1), which performs edge detection.

The discussions involved debating whether to develop a custom edge detection. After researching algorithms for edge detection (6), the team preferred the method already in use inside of the Sobel filter. So, it had made more sense to further develop it to suit the project's needs, to save resources for other sub-systems. This method generated quite a few versions after many iterations; these are discussed in the next section of the report, 'Proposed Solutions'.

4.2 Phrase Outlining

To come up with an algorithm to outline phrases, the group decided to decompose the problem and to generate functions to perform the new decomposed, smaller systems. Figure 4.2.1 shows the decomposition of outlining a phrase.

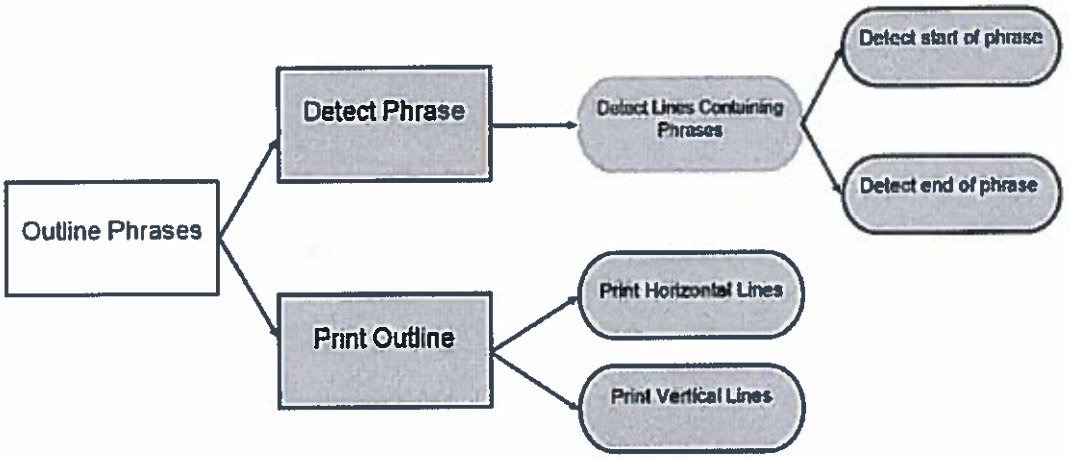


Figure 4.2.1: Problem Decomposition for Outlining Phrases

Research was the primary method used to come up with designs that would perform the phrase detection functions from Figure 4.2.1. Finding out the advantages and disadvantages of methods researched, allowed the combination of parts relevant to the project, whilst factoring in the limitations that can be caused by using certain techniques. Also, research made the major development hurdles clearer, allowing the group to know which sub-systems would require more resources and effort; this lead to less frustration as difficulties were predicted more accurately.

OCR was the original aim, and initial research around OCR showed that there were methods that could be adapted to the new project aim. This lead to the group investing more time into the research of OCR. However, research was also conducted in a couple of other areas that share algorithm elements and features. For example, techniques in object detection could have been useful if the system was designed to treat phrases as objects.

To generate concepts for printing the outlining, the group decided to adopt the 'user narrative' approach. To do this, the user's view and interaction with the system were simulated by team members, and so the designs came from what the group thought the user would expect to happen. The group predicted that the user would strongly prefer the box to be drawn seamlessly, meaning that it shouldn't fade in. This lead to the group wanting to design a function that would draw the outline in one frame, not over a couple or more. Also, from this technique, the group realised that the outlining box should contrast as much as possible to the background, a high contrast would allow the user to be quickly recognise that the system has detected the phrase. To implement the concepts created, the team would have to consider limitations based on the general system design, and so a few ideas were discarded as they were incompatible with the rest of the system.

The group felt that the use of this technique for this scenario was not particularly effective. The aim was to bring out techniques for the system to be able to print the outlining, not what the outlining should have been. So, based on previous success with research and SCAMPER, the team ultimately decided to use a mix of these two methods to come up with ways of producing a printing function.

4.3 Terminal User Interface

After using 'User Narrative' for the first time, the group attempted to use the technique again to create some ideas for the User Interface, (UI), found on the terminal. Based from the initial use of the technique, the group able to get more from the session as it was chosen this time due to it being more suited to the task.

The UI would allow the user to control which state from Figure 3.1 the system is in. Through this, the group realised that the user would want some type of performance feedback, to be able to objectively decide if the performance specification was being met. The design should also have included input feedback, so that the user can confirm that their commands had been registered and the system is still operating. If the system were to lock up, it would be preferable for some signal to have relayed this information. However, although this was ideal, the group recognised this would be difficult. If the system did hang, then without an Operating System protecting the HW, it would be difficult to terminate any process that has stalled the CPU, making it difficult to send information to the user over UART.

4.4 Hardware Optimisations

When the image processing functions were to be ported to work on the FPGA, the group had to come up with ideas to make the function more efficient by using less Hardware on the FPGA, and to make the system meet the target performance from the specification. The team had decided to carry out a brainstorming session to obtain optimisation methods, where a mind map of the ides generated was produced, shown in Figure 4.4.1. The ideas generated at this stage were used to optimise solutions used in the final design.

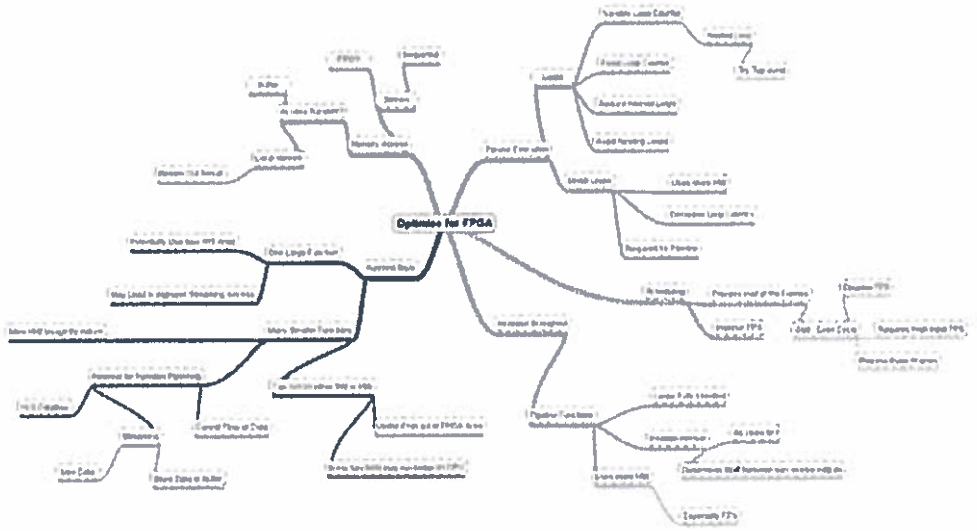


Figure 4.4.1: Mind Map from Brainstorming session on Optimisations for FPGA acceleration

When considering possible solutions, factors like memory access, loops and local variables had to be considered, as discovered by the team during the brainstorming session, displayed above in Figure 4.4.1. The main idea that had appeared, was that functions would have to access data in a sequential order. This would eventually limit the amount of viable choices the group could make when deciding on which algorithms to implement, any algorithm that required random access, would be difficult to implement and require the use of buffers.

Also, expanding on what the group learned from the interim report, possible solutions were created to decrease latency and to improve throughput. For example, to unroll a loop controlled by a variable counter, through brainstorming, the group realised that a maximum number of iterations could be set. From this, the counter could be compared to the variable through an 'if' statement, effectively making the loop perform an action with a variable loop counter.

Another great idea to come out of the session, was to only process half of the frames. This would allow the system more time to process a frame, and if the input FPS was high enough, the user shouldn't be able to notice any difference (4). The group had agreed this could have been used as a potential last resort to increase the output video stream FPS to meet the specification, if no other method had worked.

5. PROPOSED SOLUTIONS

5.1 Pre-Processing

Initially there were two ideal solutions for the pre-processing stage; the 'Sobel' filter, and a 'Binarization' filter. The Sobel filter worked by 'convoluting the grey values of the pixels with fixed 3x3 matrices', (the group's management report has a full explanation of how the Sobel filter works). The filter performed edge detection, and as characters can be viewed as edges, the filter would allow the locating of characters, once they have been detected.

The Binarization filter would compare the grey value of a pixel and compare it against a threshold value. If the grey value was above the threshold, the pixel would be set to white; if the grey value was below the threshold value, the pixel would then be set to black. The implementation for this filter was relatively easy, however it only standardised the image, and did not remove the 'inside filling' of the characters like the Sobel filter did. The use of this filter, was for that any specific two colours, the frame could be converted to a colour tone that the next sub-system could work with. Being able to only work with only one set of colours was the biggest limitation of this solution, as the threshold depended on the set of colours used.

Upon reflection, the team had decided to apply the SCAMPER method to adapt the Sobel method. The Sobel filter was compatible with many lighting conditions, discovered through testing, and could work with any pair of colours. The new solution, would instead set non-edges to white, and edges to black, to make the it compatible with the next stages of the system. Also, the parts of code retaining the grey values were discarded, as they were not required in the later stages of processing. Figure 5.1.1 displays the difference between the three solutions. The pair of blue and orange colour were chosen for the source to better highlight the need for pre-processing. The binarization filter threshold would have been set to separate blue from orange, for the example used in Figure 5.1.1.

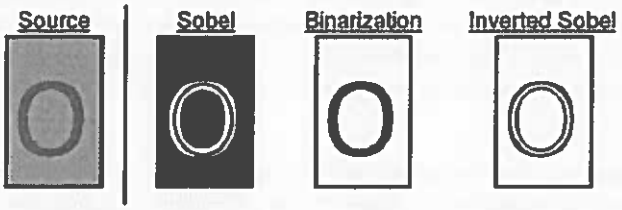


Figure 5.1.1: Optimal Results expected from Pre-Processing solutions

5.2 Phrase Detection

Initially the team researched machine learning methods and found the concept of a 'Convolutional Neural Network'. This type of neural network has many layers and hundreds of thousands of connections. However, the weights used are shared in-between layers to reduce the number of free parameters (10). This would make the implementation more manageable, and is the key advantage of using a CNN over a traditional neural network. There are many algorithms that can be implemented via a CNN, but the key idea is that is self-learning and would improve its accuracy over time.

The second method the team explored was more traditional in the field of image processing. A pixel's amplitude can be defined by its grey value. A technique known as 'Amplitude Projection', described by Pratt (9), will sum the amplitudes of each pixel in either a horizontal or vertical line. The projections are compared to a threshold which determines if the line is significant for the detection of an object. Comparing successive projections will allow the detection of an object; this method is known as 'Amplitude Segmentation'. Figure 5.2.1 shows how Amplitude Segmentation is used to detect a tank from an image. Notice that the method makes use of both horizontal and vertical projections.

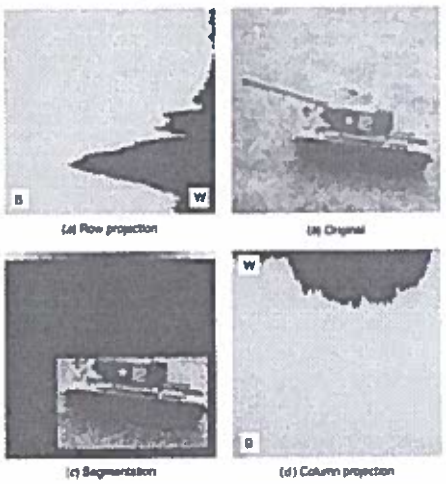


Figure 5.2.1: Amplitude Segmentation example (9: p.588).

The solution the team proposes, makes use of the 'Amplitude Projections' for only the horizontal lines. By comparing this to the threshold, noise can be factored into the value of the threshold, the system would be able to determine if a phrase occupies that specific line. Once the first line was detected, the system would set a flag to show the start of a phrase has been detected. Then the next flag, to signal the phrase has ended would be set when line did not pass the threshold test.

The problem with leaving out the vertical projections is that the start and end of the phrase on the horizontal axis would not be detected. Due to the design specification, of using a mono-coloured background, the first and last pixels, from rows with a significant projection, can be used to signal the start and end of the phrase. However, this would lead to an issue of noise being considered the start of a phrase, or for example, the edge of a paper the phrase is located on. But as these scenarios are not defined in the specification, the team believed that this was a viable solution.

The 'Stroke Width Transform' algorithm, developed by Epshtein et al (5), was found through more research. SWT works by first measuring the width between edges. This is achieved by taking the perpendicular gradient of the edge line a pixel is on. The new line is followed until the next edge is reached. Then the same steps are applied to the pixel at the end of the line, to see if both lines are similar. The length travelled is equal to the width of the stroke. To display this visually, Figure 5.2.2 shows a grid representation, design by Epshtein et al (5).

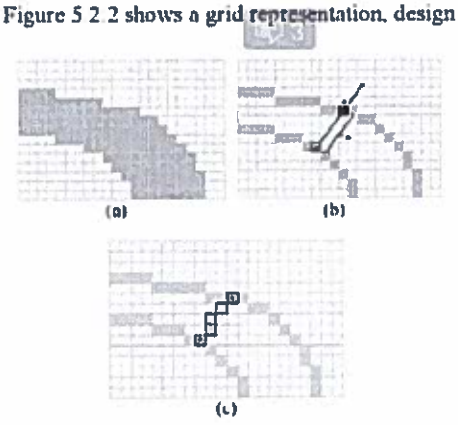


Figure 5.2.2: Illustration of the line between edges (5: p.3).

By checking for the same width size, the algorithm can detect the presence of a character. To make the system find a phrase, many factors are checked for similarities, like stroke width, letter width and character height (5).

This solution would require the use of a 'Canny' filter (6) during the pre-processing stage, to obtain an image like the one that is found in Figure 5 2 2, illustration b. Also, the team realised that when a pixel was being analysed, random access to many surrounding pixels would be necessary.

5.3 Phrase Outline Printing

The aim of the printing operation was to make it clear to the user that a phrase had been detected. The first solution the team came up with was to make use of coordinates of the corners of each phrase. This worked by comparing the current pixel's location on the grid to what coordinates the outlining box would appear on. However, this solution would require the entire frame to have been scanned beforehand, to know which pixels on the grid were included in a box, causing the use of this technique to be limited.

One of the new solutions was to set hardcoded limits before and after the first and last detected edges in the first row of a character. Whilst a line was being detected, the first and last non-white pixel location would be saved. For the leftmost vertical line of the box, the system would deduct a certain value and add the same value to the first and last pixel locations, respectively. Even though this was compatible with the system, there were a couple of key features making it not ideal. The first, was that it did not strictly meet the specification, as the box would be outlining an area greater than the area of the detected phrase. Also, it had the possible scenario where the vertical lines would have been out of the frame, if either of the actual vertical edges were close enough to the edge of the frame.

To meet the specification, an alteration to the detection sub-system had to be made. Every phrase would now have the minimum and the maximum limits updated when scanning each row. So, the lowest and highest x-coordinates, considering every row the phrase was on, would be used. This would ensure that the specification was met and no problems arose if the phrase was near the edge of the frame. However, the team had problems with data flow while implementing this solution, and so more work would have had to be done in conjunction with the chosen solution for the detection sub-system.

5.4 Terminal User Interface

The team came up with two ways that the user could interact with the system. The first method was to display a numbered list of settings in an easy to read menu. These settings would contain sub-menus, guiding the user to the required state of operation. The benefits of this solution were that it would be very easy for the user to understand how to operate the system, and that the user could navigate through the menu system easily. But the downside of this, was that it could take a long time to get to a setting if it was placed in a large nested menu.

The alternative method, was to create a command directory, and let the user directly input a command. This would allow an experienced user to switch to a setting faster than having to go through a menu system. The downside of this however, was that the learning curve would be too high for a design being used for demonstrational purposes. So, the team decided to follow through with the first solution.

Whilst the system was in a processing state, the team agreed to implement a form of feedback that would accurately notify the user of the 'exact' performance. To do this, a function was developed to measure the time between successive frames being output via VGA, measured as 'Frame Times'. A way to convert this into FPS, was to take the reciprocal.

6. FINAL DESIGN

6.1 Background and Considerations

The final design utilised a combination of a Sobel Filter and the Amplitude Segmentation method of image segmentation to attain the phrase outlining that was desired, with a frame rate of *125* FPS. The goal of a high frame rate and the physical limitations of the FPGA heavily influenced the final design of the system. Working within these constrains, the overall success of the phrase outlining system can be attributed to several pivotal design decisions. The constrains of the system and the design choices that were made are discussed below.

High Frame Rate

Attaining high frame rate boils down to the throughput of the system. Frame rate is the rate at which consecutive frames are displayed on a display. A higher throughput can translate to a higher frame rate. Throughput can be defined as rate at which the system consumes a new input or produces a new output. Under the context of the project's objective of live video processing, it is the rate at which the image processing unit of the FPGAs consumers a new pixel and produces an output pixel. Therefore, the rate at which a new output pixel is generated will directly influence the rate at which a complete frame of 1920x1080 pixels is generated. This in turn influences the frame rate. Given the direct relation between frame rate and throughput, the system was designed with the objective of maximising throughput.

Hardware Considerations

The ZYBO board provided is equipped with a wide variety of features and peripherals. The specifications that were of greater interest were the number of DSP's (Digital Signal Processing), Look-Up-Tables (LUTs) and Block Ram (BRAM).

Resource	Amount
DSP	80
BRAM	60
LUT	17600
FF	35200

Figure 6.1.1: Type and amount of Hardware available on the FPGA used.

Of greatest concern was the 60 36k BRAM that was available, detailed in figure 6.1.1. In traditional computer systems, a frame of a video can be stored in memory in a data structure such as an array. The memory can then be accessed in a sequential or non-sequential manner to be operated on by an algorithm. BRAMs, which are found on the programmable logic (PL), are the closest parallel to arrays in programmable software. They support random access to read and/or write on every clock cycle. However, a 1920x1080 video frame, where each pixel is 16 bits wide, would require **33.2 x 10^6** bits. The 1.92 x10^6 bits that the 60 BRAM on the ZYBO board offers would not be able to support this. Therefore, other methods of storing the image was required.

		Convolution Neural Network		Amplitude Segmentation		Stroke Width Transform	
Criteria	Weight	Score	W x S	Score	W x S	Score	W x S
1 Speed	6	4	24	9	54	6	36
2 Circuit Complexity	5	1	5	8	40	2	10
3 Ease of implementation	1	2	2	6	6	1	1
4 Reliability	2	10	20	7	14	7	14
5 Accuracy	1	9	9	6	6	9	9
6 Development Time	4	1	4	8	32	3	12
		Weighted total	64	Weighted total	154	Weighted total	100
Continue with Candidate?		NO		YES		NO	

Figure 6.1.2: Weighted Matrix Selection for phrase detection solutions.

To decide which solution to use for the phrase detection system, the team used the 'matrix selection' method, shown in figure 6.1.2. Amplitude segmentation was the clear winner, due to its lower development time and how fast it can potentially run at. Although, it was worth noting that the other method scored higher in reliability and accuracy, the weighting factors favoured speed and complexity more, (due to project timescale), and so ultimately, Amplitude Segmentation was chosen for the final design.

6.2 Final Design Overview

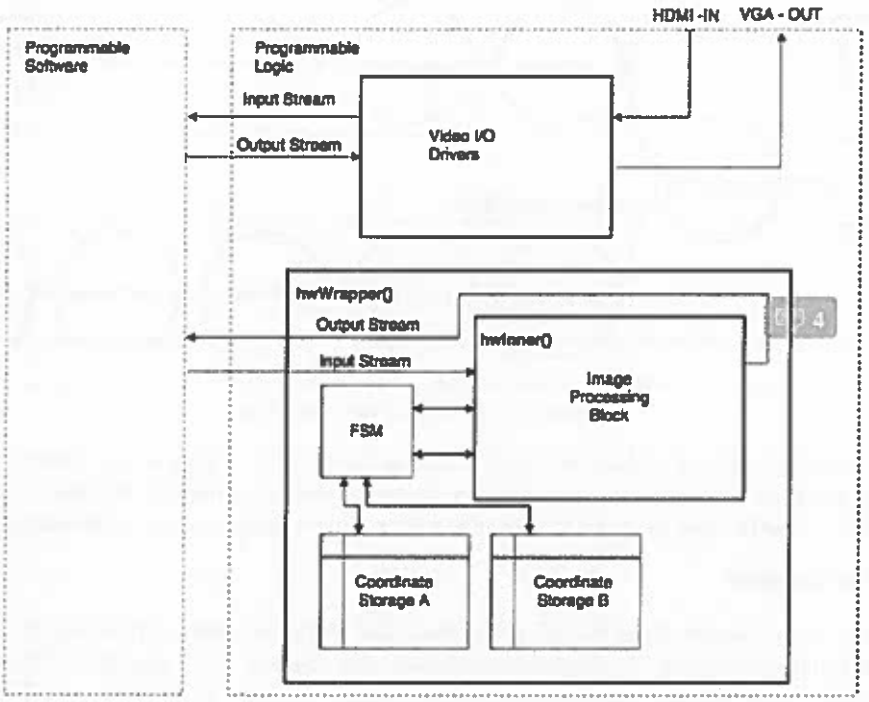


Figure 6.2.1: Final System Hierarchy

Figure 6.2.1 shows an overview of the final design of the text detection system. The bulk of the system has been implemented in PL. The portion that is implemented in Programmable Software (PS) deals only with the high-level user control via the terminal.

The input video stream from the HDMI input on the ZYBO board, was directed to the Video I/O drivers provided by Xilinx (1). The platform used was a modified version Aubert's platform (8). The Video I/O drivers also handled the output video stream to a VGA output on the ZYBO board.

The input and output video stream were accessed via two frame buffers, 'imBuff0' and 'imBuff1', that were implemented on the PS. They were linked to the Direct Memory Access (DMA) port that is set up by the Video I/O drivers. 'imBuff0' stored the input video frame while 'imBuff1' stored the output video frame. The pixels were stored in a raster scan order.

The PL section of Figure 6.2.1 also illustrates the hierarchy of the hardware functions that were implemented. At the top-level was the wrapper function 'hwWrapper'; the function controlled the reads and writes to the main memory (DDR3). Four arrays of dimensions '2x50' were declared to store the four coordinates that were used in boxing-up the detected text. These were implemented as BRAM in the PL. A Finite State Machine (FSM) controls the flow of data between the arrays and the function, 'hwInner'. The wrapper function is a key part of the system which will be elaborated upon later.

'hwInner' is the main image processing block. Its inputs are the input video stream and one of the two coordinate data storages available. Its outputs are the output video stream and a new set of coordinate data which was stored in the other coordinate data storage. The data flow diagram in Figure 6.2.2 displays an overview of the various function within 'hwInner' that operate on the input data.

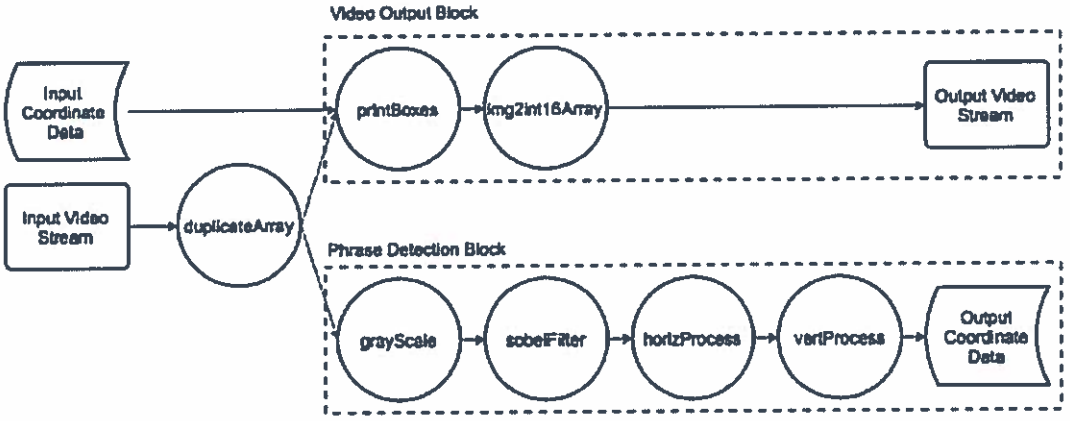


Figure 6.2.2: Dataflow Diagram hwInner

'hwInner' contains a series of hardware accelerated functions that have two major purposes; phrase detection and video output generation. However, the incoming video stream was first passed through 'duplicateArray', as shown in Figure 6.2.2. This function allows each 'chain of functions' to have their own copies of the input frame.

6.2.1 Phrase Detection

The first step was to place the frame through a Grey Scale filter which converted the frame from the RGB colour space to the Grey colour space. The Sobel Filter performed edge detection on this grey frame. The altered video stream will now contain the edge data of the frame, where an edge is denoted by a black pixel and the remainder being white pixels. The data is then passed onto 'horizProcess' and 'vertProcess'. These two functions form the backbone of the phrase extraction algorithm.

'horizProcess' performs two operations on the data. Firstly, it performs the amplitude projection computation for each row of pixels. The amplitude projection of a row is the sum of all the edge pixels in that row. Secondly, the function determines the coordinates of the leftmost and the rightmost instance of an edge in that row. The output of this function are three key values; the amplitude projection of each row, the coordinates of the leftmost pixel, and the coordinates of the rightmost pixel, for each row. This effectively reduces the size of the data from 1920x1080 to 3x1080, which is then fed into the 'vertProcess' function.

The 'vertProcess' function determines the 4 coordinates required for boxing-up the text. Using the amplitude projection values of each row of pixels, a phrase was detected when there was a cluster of rows having an amplitude projection below a pre-determined threshold, which is set to 99.2% of the maximum possible sum of the pixels as edges, in the row, (255*1080). The leftmost and the rightmost coordinates for each phrase form the vertical bounds of the image. Within the vertical bounds, the horizontal bounds were determined by the minimum of the leftmost pixels and the maximum of the rightmost pixels that corresponded to the cluster of rows detected. The output of vertProcess was a maximum of 50 quartet of coordinates that identified the location of the phrases. This was stored into the coordinate data storage arrays.

6.2.2 Video Output Generation

The function 'printBoxes' was responsible for displaying the boxes around the detected text. It does this simply by using the set of coordinates generated by 'vertProcess'. These coordinates specified the bounds of the phrases. The function then replaced the pixels that were along the perimeter of the of those bounds with a black pixel. It then output the updated frame to the output video stream which displayed the result on the external monitor.

6.3 Design Rationale

The main objective of this project was to utilise the parallel capabilities of the FPGA to accelerate the system. Guided by an understanding of the hardware limitations, the team deployed several techniques to achieve a high frame rate. The different techniques and the rationale for choosing them are discussed below.

6.3.1 Pipelining

Pipelining was used to improve the throughput of the system. Pipelining breaks down tasks into successive steps and inserts storage registers between the tasks. This enables the steps to operate on different data concurrently. This provides massive performance gains in terms of throughput since there is no one part of the system that is idling. As a result, the system can consume a new input process every N clock cycles, where N denotes number of clock cycles required to complete the longest step.

Pipelining was a perfect fit for the system since it was receiving a constant stream of data via the video input. Hence, by pipelining the system, different sections were always processing different data concurrently, thereby improving the overall throughput of the system. The system utilises both fine grain pipelining and coarse grain pipelining.

6.3.1.1 Fine Grain Pipelining

Fine grain pipelining refers to the pipelining of the individual functions of the system. This was done by specifying the pipeline directive '#pragma HLS PIPELINE II = 1' in the body of the loops whose contents were to be pipelined.

There were several conditions that had to be met for pipelining to be effective. These included no data dependencies between loop iterations, and forming perfect loops which had fixed trip counts and no logic between nested loops. The knowledge of these requirements was particularly important in designing the functions, 'printBoxes' and 'vertProcess', which read from and wrote to the coordinate data stored on BRAM.

BRAM requires one clock cycle to read or write from it (7). This implies that reading and writing from the same BRAM multiple times would require more than one cycle. In the case of 'printBoxes' and 'vertProcess' they had up to four BRAM accesses, to the same BRAM, per loop iteration. This memory access bottleneck was circumvented by implementing the data as four separate arrays namely, 'xMin', 'xMax', 'yMin' and 'yMax'. By doing so, the arrays were implemented as four different BRAMs with their own data ports, thereby supporting parallel reading and writing and hence pipelining. Originally, the data was implemented as a single array with 3 dimensions, but the software had issues resolving the indexing of the array in the manner it was declared in.

An effect of the pipeline directive was that any loop that was within the pipelined section, would be unrolled to reduce the pipeline initiation interval. This enabled the shifting methods of 'window' and 'linebuffer', that were used in the Sobel filter, to be pipelined.

Ultimately, all functions on the system achieved an Initiation Interval (II) of 1. This meant that there was only 1 clock cycle between the start of consecutive loop iterations. The consequence of this was that the functions could operate on a new data input every clock cycle. The result of pipelining a function can be observed in Vivado HLS and is shown in the Figure 6.3.1.

⊟ Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- Loop 1	2073600	2073600	2	1	1	2073600	yes

Figure 6.3.1: Latency of iteration and II from HLS synthesis for a function after being pipelined

6.3.1.2 Coarse Grain Pipelining

In coarse grain pipelining, the stages of the pipeline are the individual functions and registers are placed between the inputs and outputs of the functions. The dataflow directive '#pragma HLS DATAFLOW' was used to achieve this in the function 'hwInner'. This allowed the functions to run concurrently and operate on different data at the same time. Coarse grain pipelining enabled functions to operate as soon as a new input was available. This is contrasted with the software implementation of the functions, where one function had to complete all its operations and had produced all its outputs, before the next function could run. The result of this directive is shown in Figure 6.3.2. It can be observed that the interval of the entire 'hwInner' function is very close to the trip count of a single function within it (as seen in Figure 6.3.1), although many functions were used in sequence. This confirmed that the dataflow directive was operating as expected.

		Latency		Interval		
Instance	Module	min	max	min	max	Type
grp_hwWrapper_hwInner_fu_59	hwWrapper_hwInner	2076620	2076620	2076611	2076611	dataflow

Figure 6.3.2: Latency of Coarse Grain Pipelining implementation

Coarse grain pipelining is, by default, supported by implementing ping pong buffers between the two functions. However, since the video input was read in a streaming fashion, the data could be streamed from one function to another. This was achieved by utilising the 'hls_stream' library, which allowed the input and output ports of the functions to be defined as First-in-first-out (FIFO) streams of depth 2. The benefit of this was that the BRAM usage of FIFOs, that have a depth 2, was insignificant.

The overall effect of coarse grain and fine grain pipelining was that they system was almost completely pipelined. This meant that the throughput was maximised and as a result, the frame rate as well.

6.3.2 Double Buffer

At the top level, a double buffer system was utilised for the reading and writing of the coordinate data. The coordinate data was implemented as four separate 2 by 50 arrays but can conceptually be thought of as two data stores, each of which stores 50 quartet of coordinates. Figure 6.3.3 illustrates this technique.

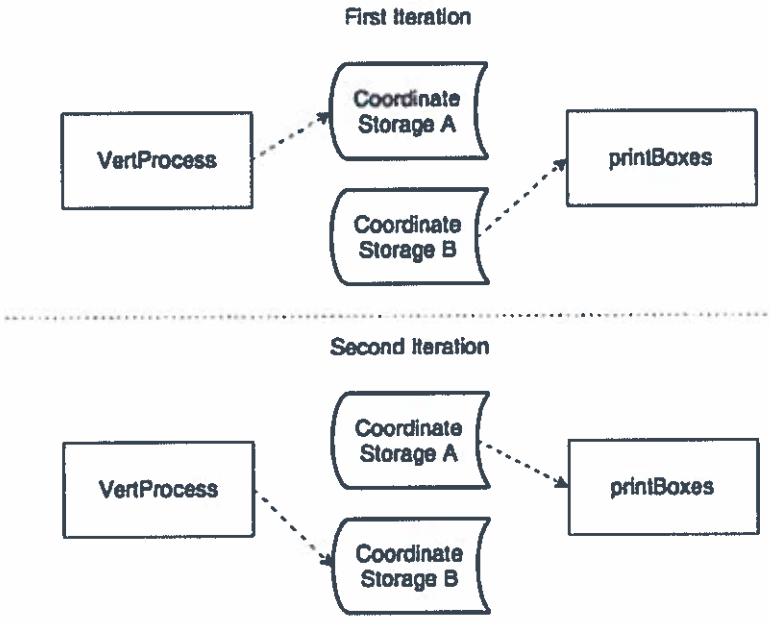


Figure 6.3.3: Visual Representation of the Double Buffer, and how it was used.

In one iteration, 'Coordinate Storage B' was used as the input to the 'hwInner' function while 'Coordinate Storage A' was used to store the output from the 'hwInner function'. More specifically, the function 'printBoxes' read from Coordinate Storage B while 'vertProcess' wrote to 'Coordinate Storage A'. Both functions reside within hwInner. In the next iteration, the roles of the data stores were switched and the 'Coordinate Storage A' became the input data store whilst the 'Coordinate Storage B' became the output data store. This back and forth ping-pong action was done indefinitely if the system was running.

The use of the double buffer was another crucial design choice. Part of the phrase detection algorithm required random access to the data, which meant that streaming the data was not an option. This was in the function 'vertProcess', where the algorithm had to determine the vertical and horizontal bounds of a phrase. The non-sequential access pattern meant that the latency of this function would be increased. The main issue with an increased latency was that the 'printBoxes' function required an unaltered video input video stream as its data source to overlay the boxes of the phrases. Although the stream could have been split using the 'duplicateArray' function and redirected to' printBoxes' as soon as the phrase detection block produced the coordinates, the latency of the dataflow path to 'printBoxes' was much shorter than the latency of the path through the phrase detection block. Therefore, by the time the data from the phrase detection block converged at 'printBoxes', 'printBoxes' would have had to buffer an entire frame of input pixels. As mentioned earlier, resource limitations inhibit this.

Therefore, by double buffering the coordinate data, the video generation block and the phrase detection block could operate simultaneously. The 'printBoxes' would overlay the boxes that were derived from the previous frame, on to the current frame. Concurrently, the phrase detection block could determine the coordinates of the boxes from the current frame, which were used in the next frame.

Implementation in HLS was not as straightforward as it seemed. Placing 'if-else' control statements within the dataflow section would disrupt the pipelining, thus the function 'hwInner' was placed within a wrapper function 'hwWrapper'. This wrapper function contained the BRAM declarations and used 'if-else' control statements to describe the double buffering action. Vivado HLS was then able to recognise this as an FSM (7), and construct the required hardware to facilitate the double buffer action with minimal loss, if any, in throughput.

6.3.3 Arbitrary Precision Types

With an understanding of how valuable hardware resources on the ZYBO board was, the team strived to design resource efficient code. Utilisation of arbitrary precision types resulted in hardware functions that utilised less resources. This came from the ability to allocate the exact number of bits required for each variable. This was achieved by following the steps prescribed by the Vivado HLS Manual (11). Regarding Figure 6.3.4, the number of LUTs used fell by 361, and Flip Flop elements used also fell by 236 after the implementation.

Before implementing Arbitrary Precision Types

Resource	Used	Total	% Utilization
DSP	3	80	3.75
BRAM	3	60	5
LUT	2078	17600	11.81
FF	1402	35200	3.98

After implementing Arbitrary Precision Types

Resource	Used	Total	% Utilization
DSP	3	80	3.75
BRAM	3	60	5
LUT	1717	17600	9.76
FF	1166	35200	3.31

Figure 6.3.4 Comparison of resource usage after implementation of Arbitrary Precision types

6.3.4 Summary of Design Choices

The use of pipelining in conjunction with the double buffering technique was crucial to the design. While a fully pipelined system meant that it had a very high throughput, the memory resources on the ZBYO were not able to support buffering of large data when the functions used had varying latencies. The use of the double buffer meant that a very small amount of data needed to be cached in PL memory, and facilitated the constant flow of data that the system required.

7. OUTCOMES

7.1 Performance

After countless of hours put into research and testing, the team eventually managed to incorporate all the techniques discussed in the previous section. Not only did the team meet the initial design target of 25 frames per second, the team went beyond that and achieved a frame rate of 125 frames per second which was 5 times as fast. Once the team identified a throughput efficient implementation by using 'HLS Dataflow', after the initial concept generation, the team never settled for less. However, this method of pixel manipulation on the input video stream was new to the team. As a result, the team had to find inspiration to accommodate the use of 'HLS Dataflow', where every pixel operation was performed in parallel. The high throughput achieved allowed for a very smooth video output. In addition, the latency of the video output was minimal and almost unnoticeable. The final design had achieved and exceeded all initial expectations in terms of performance.

7.2 Resource Usage

	Cell	Count		Cell	Count
1	BIBUF	130	24	RAM32X1D	2
2	BUFG	4	25	RAM32X1S	88
3	BUFIO	2	26	RAMB18E1	3
4	BUFR	2	27	RAMB18E1_1	2
5	CARRY4	436	28	RAMB18E1_2	2
6	DSP48E1	3	29	RAMB18E1_3	1
7	FIFO36E1	2	30	RAMB18E1_4	1
8	IDELAYCTRL	1	31	RAMB36E1	2
9	IDELAYE2	3	32	RAMB36E1_1	4
10	ISERDESE2	3	33	RAMB36E1_2	4
11	ISERDESE2_1	3	34	RAMB36E1_3	1
12	LUT1	1070	35	RAMB36E1_4	4
13	LUT2	2009	36	SRL16	4
14	LUT3	3877	37	SRL16E	668
15	LUT4	2183	38	SRLC32E	116
16	LUT5	2240	39	FDCE	1104
17	LUT6	3233	40	FDPE	531
18	MMCME2_ADV	2	41	FDR	380
19	MUXCY	172	42	FDRE	18202
20	MUXF7	242	43	FDSE	580
21	PS7	1	44	IBUFDS	4
22	RAM16X1S	176	45	IOBUF	4
23	RAM32M	60	46	OBUF	20

Figure 7.2.1 Resource Usage of Final Design

Ultimately, the team managed to compile the hardware functions within the resource constraints of the Zybo Board. Referring to Figure 7.2.1, the final design used a total of 24 BRAM blocks, 20797 Flip flops and 14612 Look Up Table (LUT) Cells. This was approximately 60% of the total resources available which suggests some room for further extensions to the design if given more time. It was particularly challenging at the start to code in the way that Vivado HLS favours, to minimise resource usage on the FPGA. Thus, the team initially struggled to get any of the hardware functions to compile. It was only after learning how to use 'HLS Dataflow' efficiently that the team managed to compile hardware functions. Implementation via 'HLS Dataflow' not only improved the frame rate tremendously, but also facilitated design of resource efficient hardware. This implementation reduced duplication of unnecessary hardware, which allowed the team to overcome the initial challenge where the design often used 20%-30% more LUTs than what was available. Having managed to limit the resource usage to that within the constraints of the Zybo board, the team had successfully met expectations.

7.3 Accuracy and Reliability

The final design attained a level of accuracy that exceeded the team's expectations. It could detect words (both lowercase and upperease) of any font, colour, or size against any mono-coloured background with an accuracy level above 99%. Figure 7.3.1 shows a capture of the board output, successfully detecting multicoloured words of different sizes and font types.

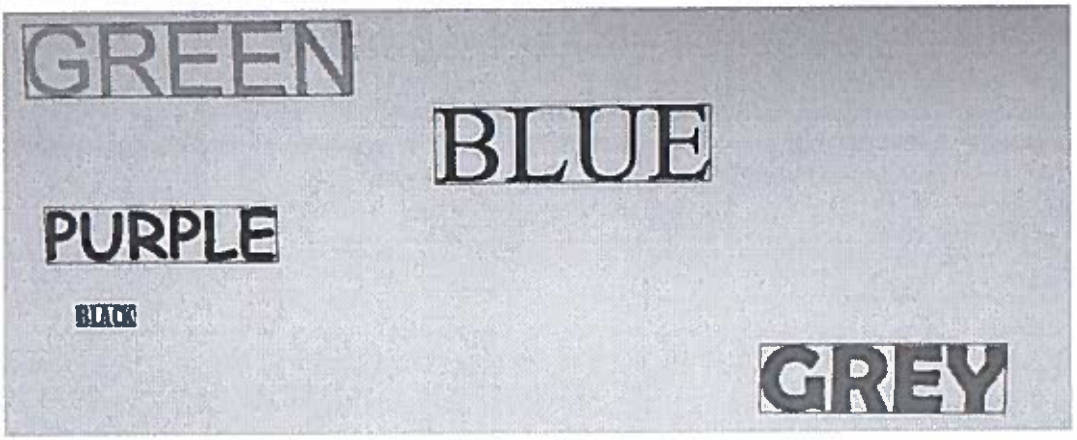


Figure 7.3.1 Successful Detection of Multicoloured words with different sizes and fonts

The team also tested the design against an input video stream coming directly from a camera. Under a reasonably lit environment, it achieved similar results to that of a digital input stream despite facing minor distortion due to noise from the environment. In addition, the system was also sophisticated enough to detect not only printed but handwritten words as well. This validates the robustness and reliability of the system. Lastly, the final design also managed to meet an operation duration of at least 30 minutes.

7.4 Limitations

Despite all the positive results produced by the design, there were inevitably still flaws and assumptions that the team had to accept while designing it.

i) Width of phrase or sentence must be thicker than a single stroke (e.g. "I")
The team incorporated both edge detection and amplitude segmentation in the detection algorithm. As such, each row of the video input stream is required to contain a certain threshold of corners (more than 3) before it recognises that as the start of a line. Hence, characters like "I" which effectively have only 2 corners per row, are deemed to be below the threshold. This was due to the team's consideration in implementing a noise rejection mechanism. The threshold was decided on empirically after several rounds of testing and evaluation. Furthermore, the team agreed that this was a fair assumption as it was uncommon for an entire sentence or phrase to only contain a single "I" or "L" character. Figure 7.4.1 below demonstrates an instance of such a limitation.

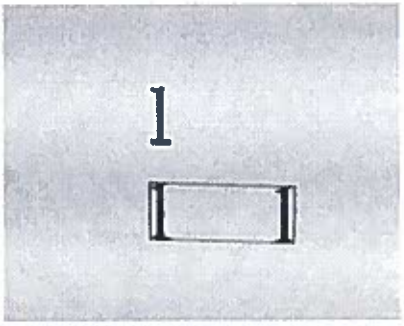


Figure 7.4.1 Unsuccessful Detection of phrases which were as thin as a single stroke

ii) Font Colours must be slightly contrasting against the background colour

Next, it is important to clarify that the system was naturally designed to only detect fonts which were different in colour to that of the background. The design pre-processed each frame with the use of a Sobel filter which also effectively converted the video input stream from RGB to grey value. As such, font colours with similar grey value to that of the background would innately be difficult for the system to recognise. Here, in Figure 7.4.2, it was observed the design was unable to identify the yellow coloured word:

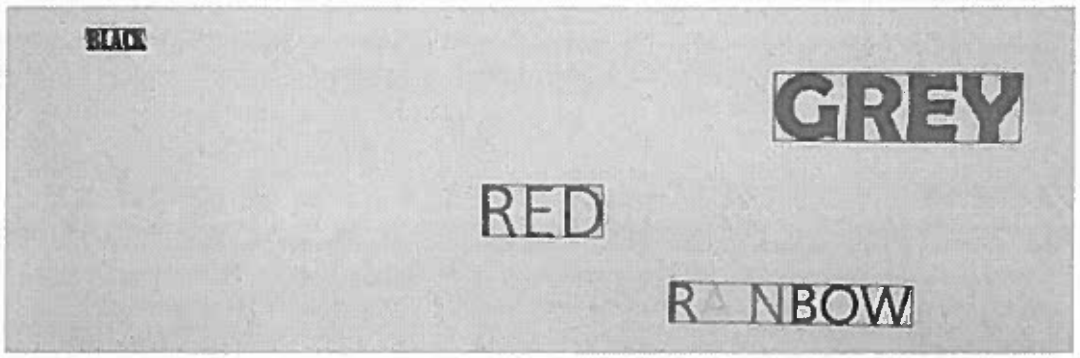


Figure 7.4.2 Unsuccessful detection of yellow coloured font against a white background

The team had identified a straightforward solution, which was to alter the threshold used in the edge detection filter. Although this implementation allowed successful detection of fonts with similar grey values, it would also often misidentify patches of shadow as fonts. Thus, lowering the threshold reduced the system's ability to work under variable lighting conditions. Hence, rather than having an inaccurate design that works for all phrase colours, the team decided to maintain the accuracy level, (about 99%), and have it work for a couple less phrase colours.

iii) Vertical limits of Phrases must not overlap

Finally, the vertical limits of each phrase must not overlap. This was an inevitable problem arising from the use of amplitude projection to detect phrases. Figure 7.4.3 demonstrates one of its flaws:

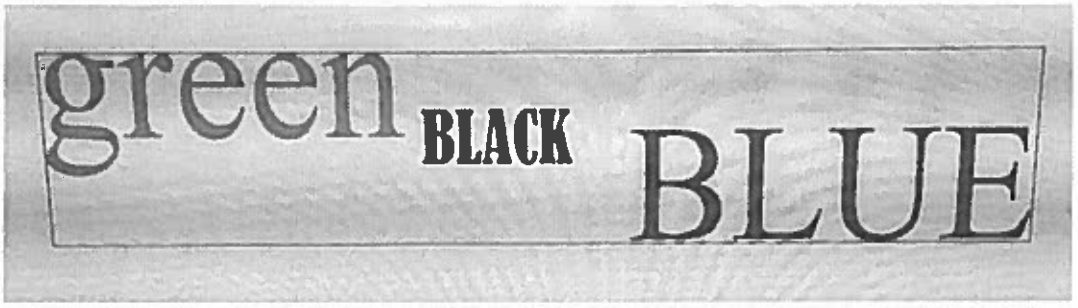


Figure 7.4.3 Unsuccessful detection of Overlapping phrases

Despite this, amplitude segmentation fits in perfectly with the use of 'HLS dataflow', as it allows for sequential memory access. Essentially, the use of amplitude segmentation was what kept the throughput high. The other alternatives proposed earlier in Section 5, involved random memory access which would reduce the throughput of the overall design. The team has chosen to assume that the video input would contain no overlapping phrases.

7.5 Suggestions for the Future

As mentioned in Section 1 of the report, the team's initial objective was to perform Optical Character Recognition. Had more time been allocated for this project, the team would strive towards achieving the original objective. The existing code would be used to pre-process and extract phrases from an image, for an OCR system. In addition to this, an Artificial Neural Network would be implemented to recognise each character and convert it to its corresponding "string" representation.

Alternatively, the current design could also be improved to work on multi-coloured and 'messy' backgrounds. More sophisticated algorithms such as 'Stroke Width Transform', as mentioned in Section 5, could be employed to detect phrases against any background.

7.6 Conclusions

In conclusion, the team has benefitted from extensive research and testing. The process taken to correctly exploit the resources and capabilities on the FPGA was challenging yet fruitful. The team also enjoyed the project and is extremely satisfied with the final design as it had exceeded expectations set in the design specification.

REFERENCES

1. Xilinx. Zybo HDMI In. [Code] Xilinx Inc. 2011.
2. Xilinx. *SDSoC Environment User Guide: Getting Started*. [PDF] Xilinx; 2015.
3. Mind Tools Editorial Team. *SCAMPER: Improving Products and Services*. Available from: https://www.mindtools.com/pages/article/newCT_02.htm [Accessed 4th May 2017].
4. Davis SW. Auditory and Visual Flicker-Fusion as Measures of Fatigue. *The American Journal of Psychology* 1955,68(4): 654-657.
5. Epshtein B, Ofek E, Wexler Y. IEEE International Conference on Computer Vision and Pattern Recognition. In: Microsoft Corporation. Detecting Text in Natural Scenes with Stroke Width Transform. San Francisco; 2010.
6. Canny J. A Computational Approach To Edge Detection. *IEEE Trans. Pattern Analysis and Machine Intelligence* 1986;8: 679- 714.
7. Karras K, Hrica J. *Designing Protocol Processing Systems with Vivado High-Level Synthesis*. Xilinx; 2014. Available from: https://www.xilinx.com/support/documentation/application_notes/xapp1209-designing-protocol-processing-systems-hls.pdf [Accessed 16th May 2017].
8. Aubert J. High Level Synthesis of accelerators material calculations based on OpenCV. Bachelor thesis. Haute Ecole Spécialisée de Suisse occidentale; 2016.
9. Pratt W K. *Digital Image Processing*. Edition 4. John Wiley & Sons;2007.
10. LeCun Y, Bengio Y. *Convolutional Neural Networks for Images, Speech, and Time-Series*. Holmdel: AT&T Bell Laboratories; 1990. Available from: http://yann.lecun.com/exdb/publis/pdf/lecun-bengio-95a.pdf [Accessed 6th May 2017].
11. Xilinx. *Vivado Design Suite User Guide: High Level Synthesis*. [PDF] Xilinx; 2015.

APPENDIX A: CODE FOR FINAL DESIGN

A.1 System Setup and Control

```
/* ------------------------------------------------------------ */
/*                  Global Variables                            */
/* ------------------------------------------------------------ */

//Display and Video Driver structs
DisplayCtrl dispCtrl;
XAxiVdma vdma;
VideoCapture videoCapt;

//Pointers (physical addr) on frame buffers
u_int16_t *pPhysFrames[DISPLAY_NUM_FRAMES];

//Class to perform address mapping translation : physical -> virtual addr
xil_lin_io addrMap;

//For the menu
pthread_mutex_t menuLock;
volatile MenuStates menuSelect = MENU_PT;


/**
 * Entry point main.
 * @param argc number of arguments.
 * @param argv arguments (as an array of strings).
 * @return status code.
 */
int main(int argc, char **argv)
{

    u_int16_t *imBuff0, *imBuff1, *imBuff2; //framebuffer access

    TIME_STAMP_INIT()

    //load the kernel module that allocates 3 contiguous frame buffers
    system("/sbin/insmod /lib/modules/4.0.0-xilinx-apf/extra/myVDMA.ko");


    //get the physical address of the framebuffers
    for(int i = 0; i < DISPLAY_NUM_FRAMES ; i++){
        //mapping of memory allocated by DMA
        pPhysFrames[i] = (u_int16_t *)getFramebufferPhysAddr(i);
        if (pPhysFrames[i] == (u_int16_t *)BAD_PHYS_ADDR){
            return EXIT_FAILURE;
        }
    }

    //perform mapping using SDSoC mmap (DMA transfer required)
    imBuff0 = (u_int16_t *)sds_mmap(pPhysFrames[0], IM_SIZE_BYTE,NULL);
    imBuff1 = (u_int16_t *)sds_mmap(pPhysFrames[1], IM_SIZE_BYTE,NULL);
    imBuff2 = (u_int16_t *)sds_mmap(pPhysFrames[2], IM_SIZE_BYTE,NULL);


    //initialize PL (IPs)
    DemoInitialize();

    bool firstLoop = true;
    MenuStates oldMenuSelect = MENU_HW;
```

```
if(pthread_mutex_init(&menuLock, NULL) != 0){
    printf("Mutex init failed\n");
    return EXIT_FAILURE;
}

//create and run a thread for the menu selection
pthread_t thread;
int rc;
std::cout << "main() : creating thread for menu selection, " << std::endl;
rc = pthread_create(&thread, NULL, threadChooseMenu, NULL);
if (rc){
    std::cout << "Error:unable to create thread," << rc << std::endl;
    return EXIT_FAILURE;
}

//main behavior
while(menuSelect != MENU_EXIT){                          //new loop when new choice
    if(menuSelect != oldMenuSelect){
        if(menuSelect == MENU_FPS){                      //display current measure
            pthread_mutex_lock(&menuLock);
            menuSelect = oldMenuSelect;                  //Stay in previous mode
            pthread_mutex_unlock(&menuLock);

            if(oldMenuSelect == MENU_SW){
                printf("SW_PERFS with last %d samples:\nMean time : %e [s]\nMean freq :
                %e [Hz]\n",
                        swAvg.getSampleNB(), swAvg.getCurrentMean(),
                        (double)1.0/swAvg.getCurrentMean());
            }
            else if (oldMenuSelect == MENU_HW){
                printf("HW_PERFS with last %d samples:\nMean time : %e [s]\nMean freq :
                %e [Hz]\n",
                        hwAvg.getSampleNB(),
                        hwAvg.getCurrentMean(),(double)1.0/hwAvg.getCurrentMean());
            }
        }
        else{
            firstLoop = true;
        }
        oldMenuSelect = menuSelect;
    }
    if (menuSelect == MENU_PT){
        if(dispCtrl.curFrame != 0) {//display framebuffer0
            DisplayChangeFrame(&dispCtrl,0);
        }
    }
    else{
        if(dispCtrl.curFrame != 1){//display framebuffer1
            DisplayChangeFrame(&dispCtrl,1);
        }
        switch(menuSelect){
            case MENU_SW:
                if(firstLoop){
                    printf("Boxing - Software implementation running ! \n");
                }
                TIME_STAMP_RESTART()
```

```
                swWrapper((u_int16_t*)imBuff0, (u_int16_t*)imBuff1);
                TIME_STAMP()
                swAvg.addSample(timeMes);
                break;
            case MENU_HW:
                if(firstLoop){
                    printf("Boxing - Hardware Implementation running ! \n");
                }
                TIME_STAMP_RESTART()
                hwWrapper((u_int16_t*)imBuff0, (u_int16_t*)imBuff1);
                TIME_STAMP()
                hwAvg.addSample(timeMes);
                break;
            case MENU_HW_C:
                if(firstLoop){
                    printf("Censorship - Implementation running ! \n");
                }
                TIME_STAMP_RESTART()
                swWrapper_censor( (u_int16_t*)imBuff0, (u_int16_t*)imBuff1 );
                TIME_STAMP()
                hwAvg.addSample(timeMes);
                break;

            default:
                break;
        }
    }
    firstLoop = false;
}
//wait for menu thread to finish
pthread_join(thread,NULL);
pthread_mutex_destroy(&menuLock);
//Stop video
VideoStop(&videoCapt);
DisplayStop(&dispCtrl);
//free mappings
addrMap.~xil_lin_io();
sds_munmap(imBuff0);
sds_munmap(imBuff1);
sds_munmap(imBuff2);

return EXIT_SUCCESS;
}
```

A.2 Coarse Grain Pipelining

```
#include "my_ocv_hw.h"
#include <stdio.h>
#include "ap_video.h"

#define ABS(x) ((x>0)? x : -x)

void hwInner(u_int16_t video_in[IM_PIXELS],u_int11_t xMin_in[60],u_int11_t
xMax_in[60],u_int11_t yStart_in[60],u_int11_t yEnd_in[60],u_int11_t xMin_out[60],u_int11_t
xMax_out[60],u_int11_t yStart_out[60],u_int11_t yEnd_out[60], u_int16_t video_out[IM_PIXELS]){

    hls::stream<u_int16_t> img_1("img_1");
    hls::stream<u_int16_t> img_1_("img_1_");
    hls::stream<u_int8_t> img_2("img_2");
    hls::stream<u_int8_t> img_3("img_3");
    hls::stream<u_int16_t> img_4("img_4");
    #pragma HLS stream depth=2 variable=img_1
    #pragma HLS stream depth=2 variable=img_1_
    #pragma HLS stream depth=2 variable=img_2
    #pragma HLS stream depth=2 variable=img_3
    #pragma HLS stream depth=2 variable=img_4

    hls::stream<u_int19_t> sum_buff;
    hls::stream<u_int11_t> min_buff;
    hls::stream<u_int11_t> max_buff;
    #pragma HLS stream depth=2 variable=sum_buff
    #pragma HLS stream depth=2 variable=min_buff
    #pragma HLS stream depth=2 variable=max_buff

#pragma HLS DATAFLOW
    imageproc::duplicateArray<u_int16_t,IM_WIDTH,IM_HEIGHT>(video_in, img_1,img_1_);
    imageproc::grayScale<u_int16_t,IM_WIDTH,IM_HEIGHT>(img_1, img_2); //video in to grayscale
    imageproc::sobelFilter<u_int16_t,IM_WIDTH,IM_HEIGHT>(img_2,img_3); //grayscale to sobel

    imageproc::horizProcessing<u_int16_t,IM_WIDTH,IM_HEIGHT>(img_3,sum_buff,min_buff,max_buff)
    ;

    imageproc::vertProcessing<u_int16_t,IM_WIDTH,IM_HEIGHT>(sum_buff,min_buff,max_buff,xMin_ou
    t,xMax_out,yStart_out,yEnd_out);

    imageproc::printBoxes<u_int16_t,IM_WIDTH,IM_HEIGHT>(img_1_,xMin_in,xMax_in,yStart_in,yEnd_
    in,img_4);
    imageproc::img2int16Array<u_int16_t,IM_WIDTH,IM_HEIGHT>(img_4, video_out);
}

void hwWrapper(u_int16_t video_in[IM_PIXELS],u_int16_t video_out[IM_PIXELS]){

    static u_int1_t ppCount;

    static u_int11_t coord_xMin_pp[2][MAX_ELEMENTS];
    static u_int11_t coord_xMax_pp[2][MAX_ELEMENTS];
    static u_int11_t coord_yMin_pp[2][MAX_ELEMENTS];
    static u_int11_t coord_yMax_pp[2][MAX_ELEMENTS];

    static u_int8_t N[2];

#pragma HLS ARRAY_PARTITION variable=N complete
#pragma HLS ARRAY_PARTITION variable=coord_xMin_pp block factor=2 dim=1
```

```
#pragma HLS ARRAY_PARTITION variable=coord_xMax_pp block factor=2 dim=1
#pragma HLS ARRAY_PARTITION variable=coord_yMin_pp block factor=2 dim=1
#pragma HLS ARRAY_PARTITION variable=coord_yMax_pp block factor=2 dim=1

//#pragma HLS RESOURCE variable=coord_buff_xMin core=RAM_2P_BRAM
//#pragma HLS RESOURCE variable=coord_buff_xMax core=RAM_2P_BRAM
//#pragma HLS RESOURCE variable=coord_buff_yStart core=RAM_2P_BRAM
//#pragma HLS RESOURCE variable=coord_buff_yEnd core=RAM_2P_BRAM

    if(ppCount == 0){

        hwInner(video_in,coord_xMin_pp[0],coord_xMax_pp[0],coord_yMin_pp[0],coord_yMax_pp[0],c
        oord_xMin_pp[1],coord_xMax_pp[1],coord_yMin_pp[1],coord_yMax_pp[1],video_out);
        ppCount = 1;
    }
    else{

        hwInner(video_in,coord_xMin_pp[1],coord_xMax_pp[1],coord_yMin_pp[1],coord_yMax_pp[1],c
        oord_xMin_pp[0],coord_xMax_pp[0],coord_yMin_pp[0],coord_yMax_pp[0],video_out);
        ppCount = 0;
    }
}
```

The last 'if-else' statement from this section of code, was how the double buffer was implemented.

A.3 Sub-system Functions

```
#ifndef MY_VIDEO_IO_H_
#define MY_VIDEO_IO_H_

#include <stdio.h>
#include <ap_int.h>
#include "img_specs.h"
#include "ap_video.h"


#define ABS(x) ((x>0)? x : -x)

#ifdef RES_720p
#define X_BOUND_MIN 5
#define X_BOUND_MAX 1040
#define Y_BOUND_MIN 0
#define Y_BOUND_MAX 720
#else
#define X_BOUND_MIN 5
#define X_BOUND_MAX 1920
#define Y_BOUND_MIN 0
#define Y_BOUND_MAX 1080
#endif


#define MAX_ELEMENTS 50

#define LINE_THRESHOLD (X_BOUND_MAX - X_BOUND_MIN)*253

typedef ap_uint<1>  u_int1_t;
typedef ap_uint<11>  u_int11_t;
typedef ap_uint<19>     u_int19_t ;



namespace imageproc {

template<typename T, u_int16_t COLS, u_int16_t ROWS>
void horizProcessing(hls::stream<u_int8_t> &img_in, hls::stream<u_int19_t> &sum_out,
hls::stream<u_int11_t> &min_out, hls::stream<u_int11_t> &max_out){
    const u_int11_t rows = ROWS;
    const u_int11_t cols = COLS;

    u_int19_t rowSum;
    u_int11_t Xmin;
    u_int11_t Xmax;

    for(u_int11_t ycoi = 0; ycoi < rows; ycoi++){
        for(u_int11_t xcoi = 0; xcoi < cols; xcoi++){
#pragma HLS pipeline II=1
            u_int8_t px;
            img_in.read(px);
            if(xcoi >= X_BOUND_MIN && xcoi < X_BOUND_MAX && ycoi >= Y_BOUND_MIN && ycoi <
            Y_BOUND_MAX){
                rowSum = rowSum + px;
                if(px == 0){
                    if(xcoi < Xmin){
                        Xmin = xcoi;
```

```
                    }
                    if(xcoi > Xmax){
                        Xmax = xcoi;
                    }
                }
            }
            if(xcoi == cols-1){
                sum_out.write(rowSum);
                min_out.write(Xmin);
                max_out.write(Xmax);
                Xmin=1281;
                Xmax =0;
                rowSum=0;
            }
        }
    }
}

template<typename T, u_int16_t COLS, u_int16_t ROWS>
void vertProcessing(hls::stream<u_int19_t> &sum_in, hls::stream<u_intll_t> &min_in,
hls::stream<u_intll_t> &max_in, u_intll_t coord_xMin[60],u_intll_t coord_xMax[60],u_intll_t
coord_yMin[60],u_intll_t coord_yMax[60]){
    const u_intll_t rows = (u_intll_t)ROWS;
    const u_intll_t cols = (u_intll_t)COLS;

    u_intll_t xMin;
    u_intll_t xMax;
    u_intll_t yMin;
    u_intll_t yMax;

    u_int1_t inLine=0;
    u_int8_t lineCount=0;

    for(u_intll_t ycoi = 0; ycoi < rows; ycoi++){
#pragma HLS pipeline II=1
#pragma HLS dependence variable=coord_xMin inter false
#pragma HLS dependence variable=coord_xMax inter false
#pragma HLS dependence variable=coord_yMin inter false
#pragma HLS dependence variable=coord_yMax inter false

        u_int19_t dataSum;
        u_intll_t dataMinX;
        u_intll_t dataMaxX;

        sum_in.read(dataSum);
        min_in.read(dataMinX);
        max_in.read(dataMaxX);

        if(dataSum < LINE_THRESHOLD && !inLine){
            inLine = 1;
            yMin = ycoi;
            xMin = dataMinX;
            xMax = dataMaxX;
        }
        else if(dataSum < LINE_THRESHOLD){ //alr in the line, just compare min max
            if(dataMinX < xMin){
                xMin = dataMinX;
```

```
            }
            if(dataMaxX > xMax){
                xMax = dataMaxX;
            }
        }
        else if(dataSum > LINE_THRESHOLD && inLine){ //just out of line, data is ready
            if(ycoi>0){
                yMax = ycoi-1;
            }
            else{
                yMax = ycoi;
            }
            inLine = 0;
            if(lineCount < MAX_ELEMENTS){
                coord_xMin[lineCount] = xMin;
                coord_xMax[lineCount] = xMax;
                coord_yMin[lineCount] = yMin;
                coord_yMax[lineCount] = yMax;
                lineCount++;
            }
        }
    }
}

template<typename T, u_int16_t COLS, u_int16_t ROWS>
void printBoxes(hls::stream<u_int16_t> &img_in, u_int11_t coord_xMin[60],u_int11_t
coord_xMax[60],u_int11_t coord_yMin[60],u_int11_t coord_yMax[60], hls::stream<u_int16_t>
&img_out){
    const u_int11_t rows = ROWS;
    const u_int11_t cols = COLS;

    u_int11_t xMin;
    u_int11_t xMax;
    u_int11_t yMin=0;
    u_int11_t yMax=0;
    u_int8_t lineCount = 0;

    for(u_int11_t ycoi = 0; ycoi < rows; ycoi++){
        for(u_int11_t xcoi = 0; xcoi < cols; xcoi++){
#pragma HLS pipeline II=1
#pragma HLS dependence variable=coord_xMin inter false
#pragma HLS dependence variable=coord_xMax inter false
#pragma HLS dependence variable=coord_yMin inter false
#pragma HLS dependence variable=coord_yMax inter false
            u_int16_t px;
            img_in.read(px);
            if(xcoi == 0 && ycoi == yMax+1){
                if(lineCount< MAX_ELEMENTS){
                    xMin = coord_xMin[lineCount];
                    xMax = coord_xMax[lineCount];
                    yMin = coord_yMin[lineCount];
                    yMax = coord_yMax[lineCount];
                    lineCount = lineCount + 1;
                }
            }
            if(ycoi >= yMin && ycoi <= yMax){
                if(xcoi >= xMin && xcoi <= xMax){
```

```
                    if(ycol == yMin || ycol == yMax){
                        img_out.write(0);
                    }
                    else if(xcol == xMin || xcol == xMax){
                        img_out.write(0);
                    }
                    else{
                        img_out.write(px);
                    }
                }
                else{
                    img_out.write(px);
                }
            }
            else{
                img_out.write(px);
            }
        }
    }
}

//Perform Sobel Operation
template<typename T, u_int16_t COLS, u_int16_t ROWS>
void sobelFilter( hls::stream<u_int8_t> &img_in, hls::stream<u_int8_t> &img_out){
    const u_int16_t rows = ROWS;
    const u_int16_t cols = COLS;

    ap_linebuffer<u_int8_t,3,IM_WIDTH>buff_A;
    ap_window<u_int8_t,3,3> buff_C;

    for(u_int16_t ycol = 0; ycol < rows+1; ycol++){
        for(u_int16_t xcol = 0; xcol < cols+1; xcol++){
#pragma HLS pipeline II=1
            u_int16_t edge;
            u_int8_t grayPx,temp;

            if(xcol < IM_WIDTH){
                buff_A.shift_up(xcol);
                temp = buff_A.getval(0,xcol);
            }
            if((xcol < IM_WIDTH) & (ycol < IM_HEIGHT)){
                img_in.read(grayPx);
                buff_A.insert_bottom(grayPx,xcol);
            }
            buff_C.shift_right();
            if(xcol < IM_WIDTH){
                buff_C.insert(buff_A.getval(2,xcol),0,2);
                buff_C.insert(temp,1,2);
                buff_C.insert(grayPx,2,2);
            }
            if( ycol <= 1 || xcol <= 0 || ycol == IM_HEIGHT || xcol == IM_WIDTH)
                edge=255;
            else{
                short x_weight = 0, y_weight = 0;
                u_int8_t i, j;
                const short x_op[3][3] = { {-1,0,1},{-2,0,2},{-1,0,1}};
                const short y_op[3][3] = { {1,2,1}, {0,0,0}, {-1,-2,-1}};
```

```
                for(i=0; i < 3; i++){
                    for(j = 0; j < 3; j++){
                        x_weight = x_weight + (buff_C.getval(i,j) * x_op[i][j]);
                        y_weight = y_weight + (buff_C.getval(i,j) * y_op[i][j]);
                    }
                }
                edge = ABS(x_weight) + ABS(y_weight);
                if(edge < 200)  edge = 255;
                else if(edge > 100)     edge = 0;
            }
            if(ycol > 0 && xcol > 0){
                u_int8_t edgeCasted = edge;
                img_out.write(edgeCasted);
            }
        }
    }
}

template<typename T, u_int16_t COLS, u_int16_t ROWS>
void duplicateArray( T fb[], hls::stream<u_int16_t> &img_copy_1, hls::stream<u_int16_t>
&img_copy_2){
    const u_int11_t rows = ROWS;
    const u_int11_t cols = COLS;
    u_int32_t i=0;
    for (u_int16_t row = 0; row < rows; row++) {
        for (u_int16_t col = 0; col < cols; col++) {
#pragma HLS pipeline II=1
            u_int16_t fbPix;
            fbPix = fb[i++];
            img_copy_1.write(fbPix);
            img_copy_2.write(fbPix);
        }
    }
}

// Convert Frame to GrayScale
template<typename T, u_int16_t COLS, u_int16_t ROWS>
void grayScale(hls::stream<u_int16_t> &img_in, hls::stream<u_int8_t> &img_out){
    const u_int11_t rows = ROWS;
    const u_int11_t cols = COLS;

    for (u_int11_t row = 0; row < rows; row++) {
        for (u_int11_t col = 0; col < cols; col++) {
#pragma HLS pipeline II=1
        u_int16_t fbPix; //get pixel from buff
        img_in.read(fbPix);
        u_int16_t rIn,bIn,gIn;
        u_int16_t gray;
        rIn = ((fbPix & 0xF800) >> (11-3));
        bIn = ((fbPix & 0x07C0) >> (6-3));
        gIn = ((fbPix & 0x003F) << 2);
        gray = (rIn * 76 + gIn * 150 + bIn * 29 + 128) >> 8;
        img_out.write((u_int8_t)gray);
        }
    }
}
```

```
// Convert Image to Frame
template<typename T, u_int16_t COLS, u_int16_t ROWS>
void img2int16Array(hls::stream<u_int16_t> &img,T fb[ROWS*COLS]){
    u_int32_t i =0; //iterate over buffer
    const u_int11_t rows = ROWS;
    const u_int11_t cols = COLS;

    for (u_int11_t row = 0; row < rows; row++) {
        for (u_int11_t col = 0; col < cols; col++) {
#pragma HLS pipeline II=1
        u_int16_t inPix;
        img.read(inPix);    //get pixel from stream
        fb[i++] = inPix;    //write pixel to buff
        }
    }
}

#endif /* MY_VIDEO_IO_H_ */
```

Group no 5 - Real Time Phrase Detection - Chia Hossain Lua

GRADEMARK REPORT

FINAL GRADE

/100

GENERAL COMMENTS

Instructor

PAGE 1

PAGE 2

PAGE 3

PAGE 4

PAGE 5

Comment 1

Nice layout of the mapping of the various functions

PAGE 6

PAGE 7

PAGE 8

PAGE 9

PAGE 10

PAGE 11

Comment 2

nice approach

PAGE 12

PAGE 13

Comment 3

I liked that fact that you spent time to research this

PAGE 14

PAGE 15

PAGE 16

Comment 4

nice design that provides a good overview of the designed system.

PAGE 17

PAGE 18

Comment 5

not true. The BRAMs are dual ported, so you can have two independent accesses per cycle

PAGE 19

Comment 6

Very nice design approach

PAGE 20

PAGE 21

PAGE 22

PAGE 23

PAGE 24

PAGE 25

Comment 7

Very nice report. It goes though all the major design choices, and elaborates on the important aspects of the project. It shows a deep understanding of the hardware design process, and how to use the tool.

Well done

PAGE 26

PAGE 27

PAGE 28

PAGE 29

PAGE 30

PAGE 31

PAGE 32

PAGE 33

PAGE 34

PAGE 35

PAGE 36

PAGE 37